RAI-818: INTELLIGENT MOBILE ROBOTICS

Bug Algorithms – Part I

- **General Concepts (Bug 0)**
- **Bug 1**
- **Bug 2**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:
DR. KUNWAR FARAZ AHMED KHAN

Bug Algorithms

- Special Bugs?

What's Special About Bugs

- Many planning algorithms assume *global knowledge*
- Bug algorithms assume only *local knowledge of the environment* and a *global goal*
- Bug behaviors are *simple*:
  - Follow a wall (right or left)
  - Move in a straight line toward goal
- Bug 1 and Bug 2 assume essentially tactile sensing
- Tangent Bug deals with finite distance sensing

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts

A Few General Concepts

- Workspace W
  - $\Re(2)$ or $\Re(3)$ depending on the robot
  - could be infinite (open) or bounded (closed/compact)
- Obstacle WO_i
- Free workspace $W_{free} = W \mid \cup_i WO_i$

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts

A Few General Concepts

- *known direction to goal*
  - robot can measure distance $d(x, y)$ between pts x and y
- *otherwise local sensing*
  - walls/obstacles & encoders
- *reasonable world*
  - finitely many obstacles in any finite area
  - a line will intersect an obstacle finitely many times
  - workspace is bounded

$W \subset B_r(x),\ r < \infty$

$B_r(x) = \{ y \in \Re(2) \mid d(x, y) < r \}$

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0

Bug 0 Algorithm

- *known direction to goal*
- *otherwise local sensing*
  - walls/obstacles & encoders

- *Some notation*
  - q_{start} and q_{goal}
  - *hit point* $_qH_i$
  - *leave point* $_qL_i$
  - A *path is a sequence of hit/leave* pairs bounded by q_{start} and q_{goal}

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0

Bug 0 Algorithm

- *known direction to goal*
- *otherwise local sensing*
- *walls/obstacles & encoders*

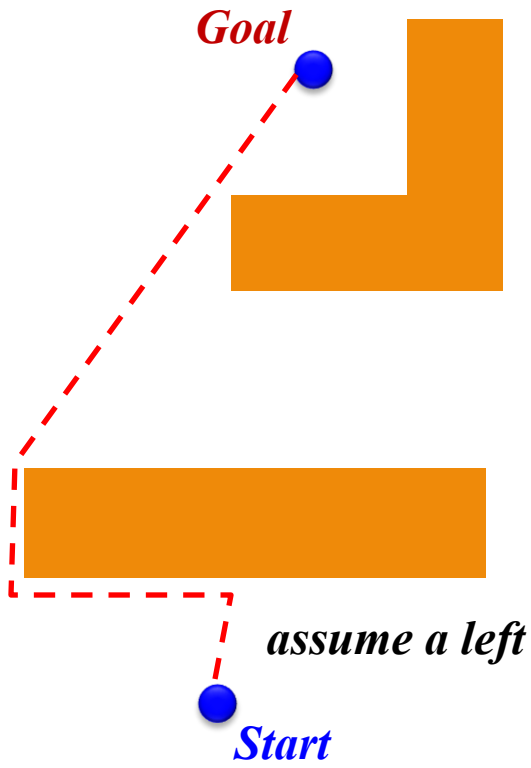
- *head towards goal*
- *Follow obstacle until you can head towards goal again*
- *continue*

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0

Bug 0 Algorithm



- *head towards goal*
- *Follow obstacle until you can head towards goal again*
- *continue*

assume a left turning robot

OK ?

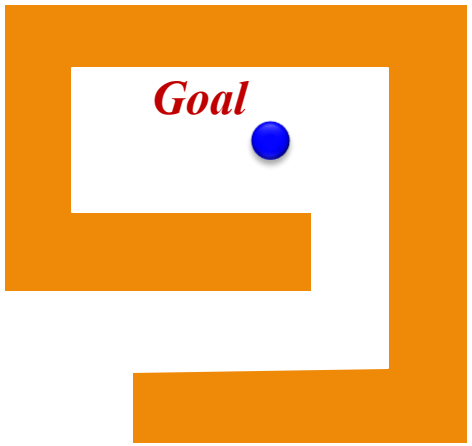
The turning direction might be decided beforehand…

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0

Bug 0 Algorithm



- *head towards goal*
- *Follow obstacle until you can head towards goal again*
- *continue*

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- A Better Bug !

A Better Bug

But add some geometry

- *known direction to goal*
- *otherwise local sensing*
- *walls/obstacles & encoders*

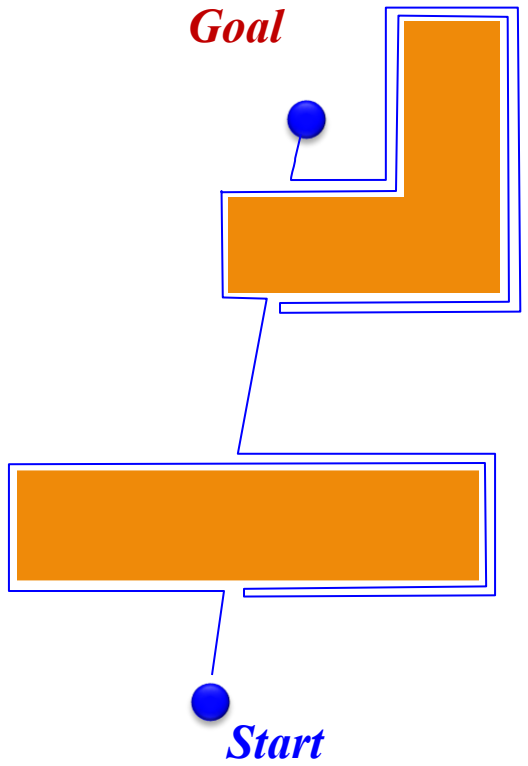
improvement ideas?

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- **Bug 1**

Bug 1 Algorithm



add some computing power

- *known direction to goal*
- *otherwise local sensing*
- *walls/obstacles & encoders*

- *head towards goal*
- *if an obstacle is encountered circumnavigate it and remember how close you reach to the goal*
- *return to that closest point (by wall-following) and continue*

Defining futures

Bug Algorithms

Bug 1 More Formally

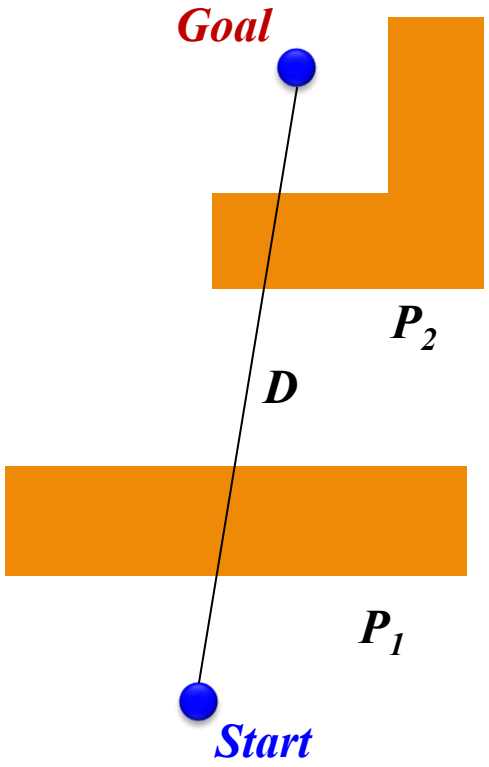
- let ${}_qL_0 = q_{start}$; $i = 1$
- repeat
  - repeat
    - from ${}_qL_{i-1}$ move toward q_{goal}
  - until goal is reached or obstacle encountered at ${}_qH_i$
  - if goal is reached, exit
  - repeat
    - follow boundary recording pt ${}_qL_i$ with shortest distance to goal
  - until q_{goal} is reached or ${}_qH_i$ is re-encountered
  - if goal is reached, exit
  - Go to ${}_qL_i$
  - if move toward q_{goal} moves into obstacle
    - exit with failure
  - else
    - $i=i+1$
  - continue

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- **Bug 1**
  - Pseudo code
  - **Bounds**

Bug 1 - Path Bounds



What are upper/lower bounds on the path length that the robot takes?

D = *straight-line distance from start to goal*

P_i = *perimeter of the i th obstacle*

Lower bound:
What's the shortest distance it might travel?

$$D$$

Upper bound:
What's the longest distance it might travel?

$$D + 1.5 \Sigma P_i$$

What is an environment where your upper bound is required?

Bug Algorithms

Bug 1 – Completeness

An algorithm is complete if, in finite time, it finds a path if such a path exists or terminates with failure if it does not.

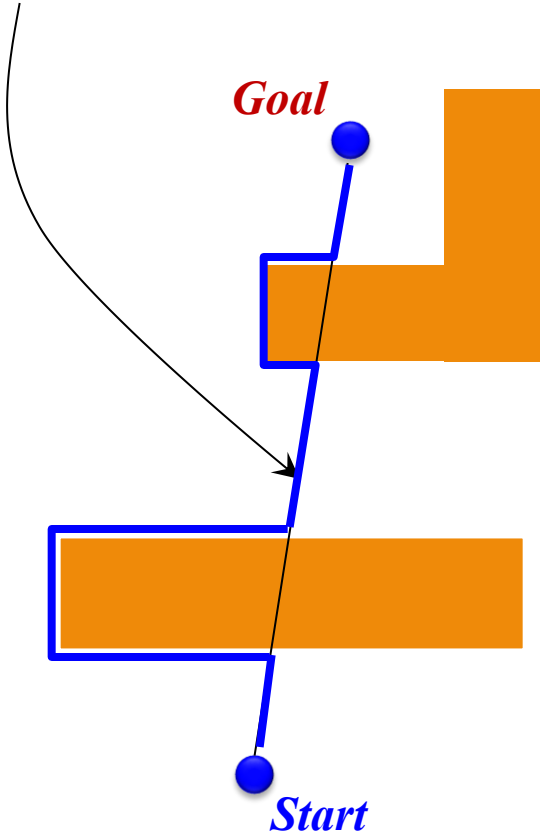
- Suppose BUG 1 were incomplete
  - Therefore, there is a path from start to goal
    - By assumption, it is finite length, intersects obstacles a finite number of times.
  - BUG 1 does not find it
    - Either it terminates incorrectly, or, it spends an infinite amount of time
    - Suppose it never terminates
      - but each leave point is closer to the obstacle than corresponding hit point
      - Each hit point is closer than the last leave point
      - Thus, there are a finite number of hit/leave pairs; after exhausting them, the robot will proceed to the goal and terminate
    - Suppose it terminates (incorrectly)
      - Then, the closest point after a hit must be a leave where it would have to move into the obstacle
      - But, then line from robot to goal must intersect object even number of times (Jordan curve theorem)
      - But then there is another intersection point on the boundary closer to object. Since we assumed there is a path, we must have crossed this pt on boundary which contradicts the definition of a leave point.

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Another Step Forward

Another Step Forward

Call the line from the starting point to the goal the m-line



Bug 2 Algorithm

head towards the goal on the m-line

if an obstacle is in the way, follow it until you encounter the m-line again

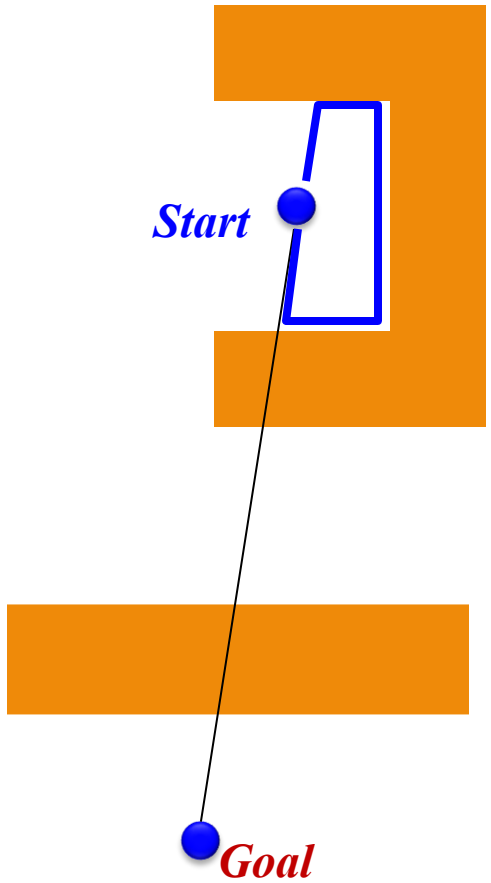
leave the obstacle and continue toward the goal

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Another Step Forward ???

Another Step Forward ???



Bug 2 Algorithm

head towards the goal on the m-line

if an obstacle is in the way, follow it until you encounter the m-line again

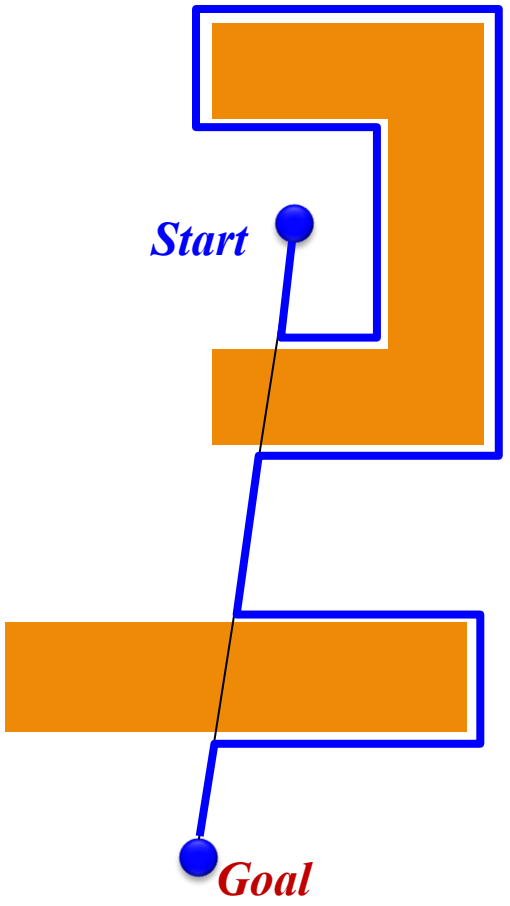
leave the obstacle and continue toward the goal

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2

Bug 2 Algorithm



head towards the goal on the m-line

if an obstacle is in the way, follow it until you encounter the m-line again closer to the goal

leave the obstacle and continue toward the goal

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- **Bug 2**
  - **Pseudo code**

Bug 2 Algorithm

Input
A point robot with a tactile sensor

Output
A path to q_{goal} or a conclusion that no such path exists

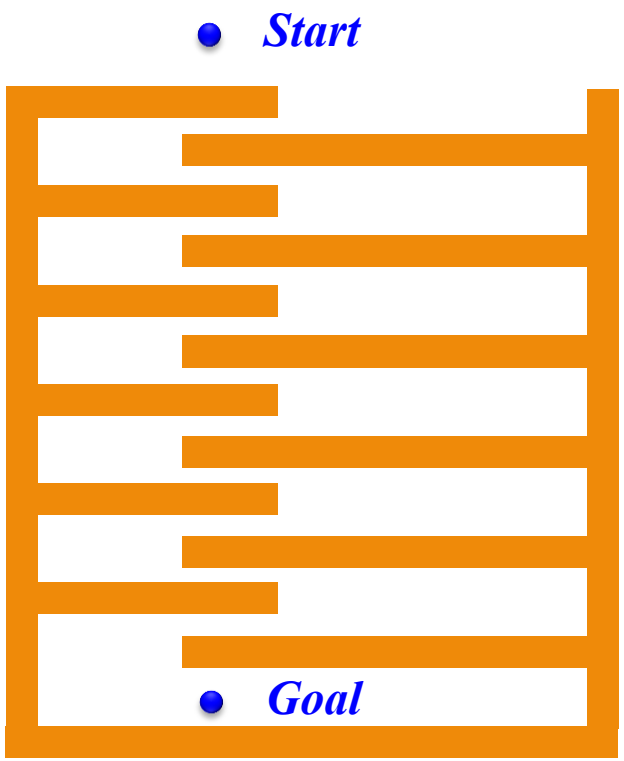
1: **while** true **do**
2: **repeat**
3: from q_{i-1}^{L}, move toward q_{goal} along the m-line
4: **until** q_{goal} is reached **or** an obstacle is encountered at *hit point* q_i^H
5: turn left (or right)
6: **repeat**
7: follow boundary
8: **until**
9: q_{goal} is reached **or**
10: q_i^H is re-encountered **or**
11: m-line is re-encountered**,** at a point m such that
12: $m \neq q_i^H$ (robot did not reach the hit point)
13: $d(m, q_{goal}) < d(m, q_i^H)$ (robot is closer), and
14: if robot moves toward goal , it would not hit the obstacle
15: **if** q_i^H is reached **then**
16: Conclude q_{goal} is not reachable and exit
17: **end if**
18: Let $q_{i+1}^{L} = m$
19: increment i
20: **end while**

Defining futures

Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
  - Pseudo Code
  - Path Bounds

Bug 2 : Path Bunds



What are upper/lower bounds on the path length that the robot takes?

***D* = straight-line distance from start to goal**

P_i = perimeter of the *i th* obstacle

Lower bound:
What's the shortest distance it might travel?

D

Upper bound:
What's the longest distance it might travel?

$$D + \frac{1}{2} \sum_i n_i P_i$$

n_i = # of *m-line* intersections of the *i th* obstacle

Defining futures

Bug Algorithms

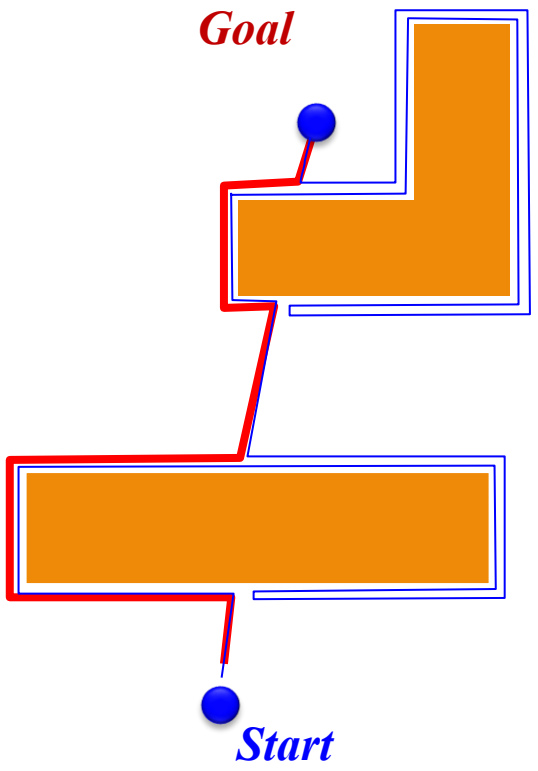
- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
  - Pseudo Code
  - Path Bounds
  - Head to Head Bug1 vs Bug2

Head to Head Comparison

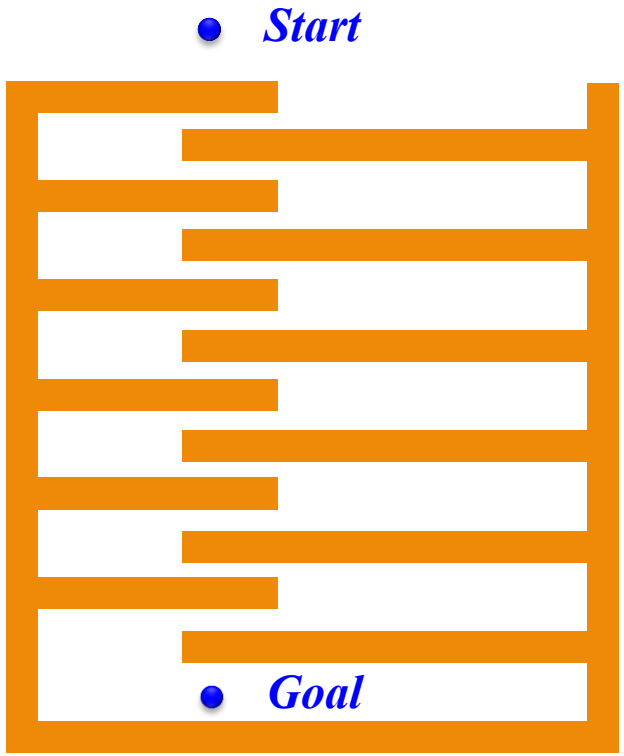
Is bug 2 better than bug1 ???

Draw worlds whereBug 2 does better than Bug 1 (and vice versa)

Bug 2 beats Bug 1



Bug 1 beats Bug 2



Defining futures

Bug Algorithms

Head to Head Comparison

- **BUG 1 is an *exhaustive search algorithm***
  - **it looks at all choices before committing**
- **BUG 2 is a *greedy algorithm***
  - **it takes the first thing that looks better**
- **In many cases, BUG 2 will outperform BUG 1, but**
- **BUG 1 has a more predictable performance overall**

Defining futures